



Sandya Mannarswamy

Welcome to CodeSport! In this issue, we will discuss the solutions to some of the challenging questions featured in last month's special New Year edition.

Many readers have sent in their answers to the questions we featured in the last edition of *LFY*. Some have requested that we keep the challenging questions open for the readers to come up with the solutions themselves. So let's look at a few of the questions from last month's column and discuss approaches to solving them instead of providing the complete solutions themselves. Congratulations to our readers Madhav Kumar, Vinay Saxena and Ravi Kiran for getting the correct answers to the maximum number of questions.

Minimum and maximum weighted cycles

To recall our first question from last month's column: given an undirected graph $G(V, E)$ where V represents the set of vertices and E represents the set of edges, and a weight function $c(u, v)$ that associates a non-negative weight with the edges of the graph, we were asked to come up with an algorithm to find the minimum weighted cycle.

We had discussed Dijkstra's algorithm to find the shortest path (minimum weighted) between two vertices in a given graph. Can we somehow use Dijkstra's algorithm to find the minimum weighted cycle involving an edge (u, v) in the graph? Recall that in order for the edge (u, v) to be part of a cycle, there should be another path between the vertices (u, v) which does not involve the edge (u, v) . We can use Dijkstra's shortest path to find the shortest path between vertices (u, v) . Let us denote this shortest path as P_2 , whereas the direct path from vertex 'u' to vertex 'v' using the edge (u, v) is denoted as P_1 . The minimum weight of the cycle which involves the edge (u, v) is the sum of values P_1 and P_2 . For each edge $e \in E$, we can compute the minimum weight cycle $C(e)$ involving that edge. And the minimum over all such cycles is the minimum weighted cycle of the graph. I leave it to the reader to come up with the complete solution.

The second part of the problem asked readers to come up with an algorithm to find the maximum weighted cycle in the graph. Recall that there is no polynomial time algorithm to find the longest (maximum weighted) path between two vertices in a given directed graph. Can you extend this argument to the problem of finding the maximum weighted cycle in a graph using a formulation similar to the minimum weighted cycle we discussed earlier?

Finding the smallest 20 numbers in an array

To recap the second problem from last month's column, given an array of N unsorted integers, you were asked to find the smallest 20 integers from that array. The hint given was to use the linear time algorithm for median finding. Note that the question asks you to find the 20 smallest integers from the array, but you do not have to order these 20 numbers from the smallest to the largest. You can find the median in linear time. Now if you partition the array using the median (remember the partition procedure from the quick sort algorithm the we discussed in a previous column?), you know that all elements partitioned below the median are smaller than the median and all elements partitioned above the median are larger than the median. Assuming that N is 31, if you find the median and partition around the median, you know that $N/2$ elements are smaller than the median; therefore, 15 elements which are partitioned below the median are smaller than the median. You need to find the remaining four smallest integers from the upper side of the partition by repeating the same procedure of finding the median and partitioning using the median. I leave it to you to use this idea to arrive at the complete solution.

Finding the 'k' minimum elements

Another question posted in last month's column was the problem of doing 'k' extract-minimum operations repeatedly on a given array of N unsorted integers. The question assumes that the value of k is not known in advance. The naive solution is to sort the array in $O(N \log N)$ and then extract the first k elements in the sorted array. However, the first part of the problem was to come up with an algorithm that can do this in $O(Nk)$ time complexity. You know that finding the minimum in a given array of N integers can be done in $O(N)$ and you can repeatedly perform 'find_minimum' k times to arrive at a solution whose complexity is $O(Nk)$. The second part of the question was slightly tougher. It asked readers to find an algorithm for this problem whose complexity is $O(N + k \log N)$. How do you do this?

The trick is to recognise that you perform 'k' extract minimum operations. Note that in order for the overall complexity to be $O(N + k \log N)$, each of the extract minimum should be taking $O(\log N)$. What is the data structure that performs 'extract minimum' with a